



Factory

Development as a reproducible process

How we turn messy problems into shipped, verified, documented improvements - without losing context.

epics

review loops

verification gates

observability

Starter deck / 2026-05-19

Why Factory exists

The bottleneck was never imagination. It was execution speed.

Before AI, many valuable engineering tasks were technically possible but economically irrational: writing thousands of tests, exhaustively hardening edge cases, replaying failures, documenting every pattern, and reviewing every boring change with full attention.

Compress the loop

Move from idea to patch to verification in minutes, not days.

Keep control

Use tests, specs, gates, traces, and artifacts to constrain AI-generated work.

Scale the method

Capture the process so more people can operate the same factory.

Factory = a way to turn AI speed into reliable software delivery

What Factory means by reproducible

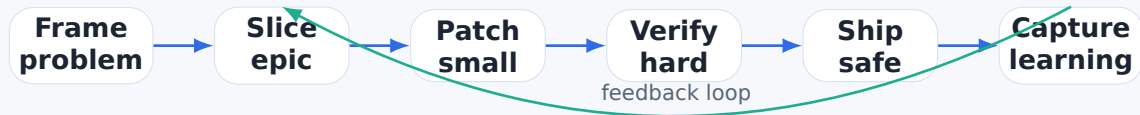
Factory does not claim that every software outcome is bit-for-bit reproducible.

It aims to make the **development process** reproducible:

- how we frame ambiguous problems;
- how we slice work into epics;
- how we preserve context and decisions;
- how we verify changes;
- how we capture reviewer feedback;
- how we turn incidents and fixes into reusable knowledge.

Reproducible process, bounded system, evidence-driven confidence

The process in one sentence



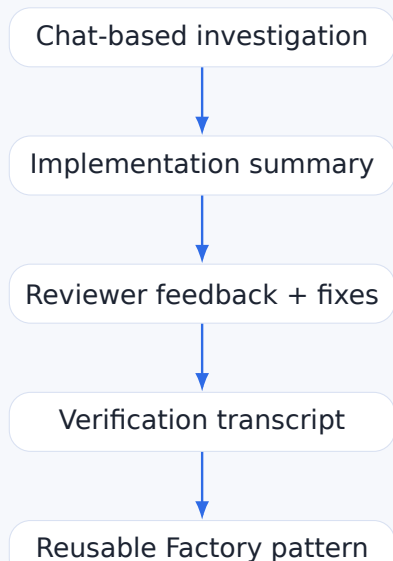
Operating principle

Every change should leave behind enough evidence for a future engineer to understand what changed, why, how it was verified, and what remains risky.

Anti-goal

Do not turn development into ceremony. Factory documents the shortest reliable path from ambiguity to confidence.

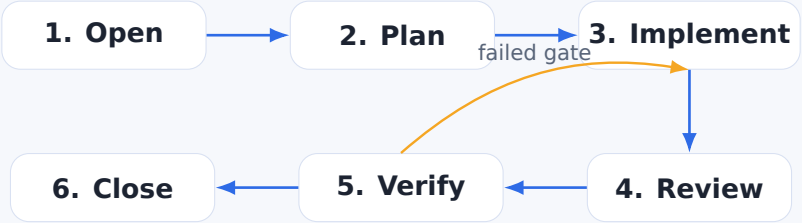
Factory converts tacit work into explicit assets



Outputs we want

- Playbooks for recurring work
- Checklists for risky changes
- Example PRs with narrative
- CI/gate design notes
- Debugging recipes
- Onboarding modules

A Factory epic has a standard shape

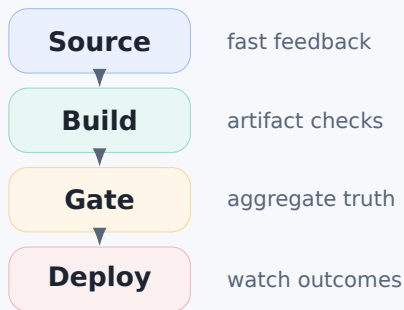


Part	What gets captured
Problem	Symptom, operator impact, current evidence, constraints.
Plan	Small slices, risk boundaries, verification strategy.
Closeout	Files changed, tests run, unresolved follow-ups, rollout notes.

Verification is part of the product, not a cleanup step

Factory asks for evidence at each layer

- Static checks: format, lint, type checks
- Unit and widget tests
- Integration and fixture regressions
- Artifact sanity checks
- Deployment gates and post-deploy observation



Rule of thumb: if a reviewer can ask “how do we know?”, the Factory artifact should already answer it.

Review loops are structured, not adversarial

Reviewer role

Find hidden risk, force precision, and turn vague confidence into explicit evidence.

Implementer role

Respond with scoped patches, verification, and concise summaries.

Factory role

Preserve the pattern: what kind of issue was caught, how it was fixed, and how it becomes a future checklist item.

Outcome

Every review improves both the code and the development system.

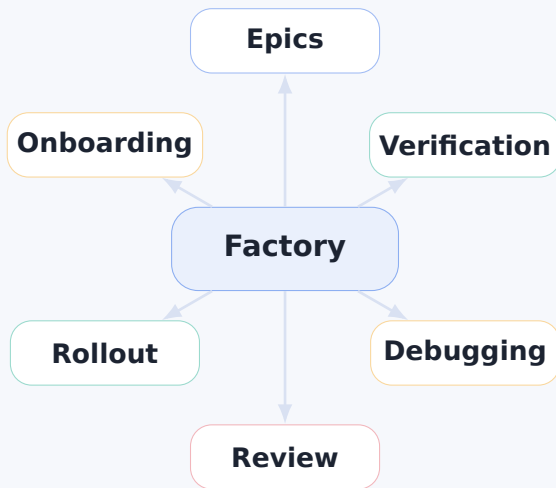
Instrumentation-first debugging

When a system behaves strangely, Factory prefers visibility over guessing.



Examples: timing breakdowns, trace diagnostics, synthetic/no-trace classifications, debug endpoints, stale artifact flags, deployment gate status files.

Factory knowledge map



First content tracks

Track	Starter artifact	Audience
Epic lifecycle	How to run an implementation epic from symptom to closeout	Engineers, tech leads
Verification gates	Artifact-driven CI gates and deployment safety	Platform, DevOps, reviewers
Debugging playbooks	Instrumentation-first trace/debug method	Backend, SRE, support
Review patterns	Turning reviewer comments into reusable checks	Everyone shipping code
Rollout notes	How to ship risky changes with observability	Leads, operators

Start with real examples. Generalize only after the pattern has survived contact with production

Suggested launch plan

1. Seed

Create a short deck, a README, and 2-3 concrete case studies from recent work.

1

shared vocabulary

2. Practice

Use Factory language during active epics: plan, evidence, gate, closeout, follow-up.

3

starter tracks

3. Spread

Turn successful epics into brown bags, onboarding docs, and reusable templates.

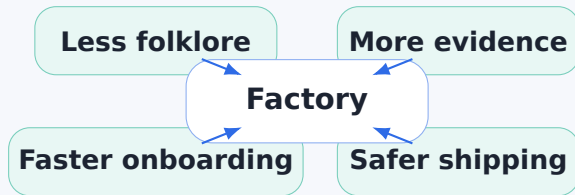
30 min

first internal talk

ongoing

living playbook

What good looks like



The end state is not “more documentation”.
It is a development process that teaches itself.

Factory

Make the way we build more inspectable, repeatable, and reliable.

next: turn this deck into the Factory README and first playbook